QuillAudits

AUDIT REPORT

February 2026

For

CHIMERA WALLET

Table of Content

Executive Summary

Project Name	CEXT
Protocol Type	ERC20 Token
Project URL	https://chimerawallet.com/
Overview	ChimeraExchangeToken is an ERC20-compliant token built using OpenZeppelin's secure and widely adopted implementations. The contract integrates ERC20Permit (EIP-2612), enabling gasless approvals threw off-chain signatures. A fixed total supply of 1,000,000,000 tokens is minted once during deployment and assigned to a specified recipient. The contract does not include minting, burning, or privileged administrative functions, resulting in a simple and low-risk token design.
Audit Scope	The scope of this Audit was to analyze the CEXT Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Outlogic-Sagl/CEXT/tree/master
Contracts in Scope	ChimeraExchangeToken.sol
Branch	master
Commit Hash	7499e4153ae9527737fdc6d8ef9d2a62e2fa7d0b
Language	Solidity
Blockchain	Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	13th February 2025
Updated Code Received	13th February 2025
Review 2	13th February 2025
Fixed In	be7db8a8f49e37c04c90d1312d2a771803bdff95

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity

Critical	0(0.0%)
High	0(0.0%)
Medium	0(0.0%)
Low	0(0.0%)
Informational	2 (100.0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	0	0	0	0	**2**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Redundant Inheritance of ERC20 Contract	Informational	Resolved
2	Compiler Version Not Fixed	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Informational Issues

Redundant Inheritance of ERC20 Contract

Resolved

Path

ChimeraExchangeToken.sol

Description

The ChimeraExchangeToken contract explicitly inherits from both ERC20 and ERC20Permit, while ERC20Permit already inherits from ERC20. This results in redundant inheritance of the ERC20 base contract. Although Solidity's linearized inheritance ensures only one instance of ERC20 is used at runtime, the redundancy reduces code clarity and slightly increases maintenance overhead.

Impact

Low

Likelihood

Low

Recommendation

Remove the explicit inheritance of ERC20 and inherit only from ERC20Permit, while still passing the ERC20 constructor parameters.

Compiler Version Not Fixed

Resolved

Path

ChimeraExchangeToken.sol

Description

The contract specifies a floating Solidity pragma version using ^0.8.27. Floating pragmas allow the compiler to use any compatible version within a range, which may lead to inconsistent compilation results across different environments. Different compiler versions can introduce subtle behavior changes, new warnings, or bugs, potentially affecting reproducibility and auditability.

Impact

Low

Likelihood

Low

Recommendation

Use a fixed compiler version

Functional Tests

Some of the tests performed are mentioned below:

- Verified that the total token supply is correctly minted to the specified recipient during contract deployment.
- Confirmed that the total supply value matches the expected amount based on token decimals.
- Tested standard ERC20 transfer functionality to ensure balances update correctly.
- Verified that transfers revert when the sender has insufficient balance.
- Tested the approve function to confirm allowances are set correctly.
- Validated that transferFrom works as expected when sufficient allowance is provided.
- Confirmed that transferFrom reverts when allowance is insufficient.
- Verified that ERC20Permit allows approvals using a valid off-chain signature.
- Tested that the permit function correctly updates allowances without requiring on-chain approval.
- Confirmed that permit reverts when the provided signature deadline has expired.
- Verified that permit reverts when the recovered signer does not match the token owner.
- Confirmed that the nonce value increments correctly after each successful permit execution.
- Verified that the DOMAIN_SEPARATOR value is correctly generated according to EIP-712.
- Tested zero-value transfers to ensure they behave according to the ERC20 specification.
- Verified that transfers to the zero address correctly revert as expected.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC20.sol	Library/ Base Contract (OpenZeppelin)	Core token logic (balances, transfers, allowances, total supply)	Correct implementation of ERC20 standard invariants; OpenZeppelin audit quality	N/A
ERC20Permit.sol	Library/ Extension (OpenZeppelin)	Gasless approvals via EIP-2612 signatures	Correct implementation of permit functionality; proper nonce management and signature validation	Signature replay attacks if improperly implemented; frontrunning of permit transactions
Solidity Compiler	Tool	Compilation to bytecode	Compiler version ^0.8.27 has no critical bugs; default settings provide adequate security	Compiler bugs could introduce vulnerabilities; older versions lack latest optimizations and security features

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
ChimeraExchangeToken .sol	constructor()	**What this function can do** • Mint the full token supply (1,000,000,000 CEXT) • Assign entire supply to specified recipient address • Initialize token metadata (name: “Chimera Exchange Token”, symbol: “CEXT”) • Initialize ERC20Permit functionality with permit domain name **What this function cannot/should not do** • Mint again after deployment (no mint function exists) • Assign tokens to multiple addresses (single recipient only) • Be called again after deployment • Change token name, symbol, or total supply • Validate recipient address (accepts zero address or any address) **Main invariant(s)** Total supply is fixed at deployment **Access level** Implicit restriction (once at deployment)

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
ChimeraExchangeToken.sol	ERC20 (CEXT)	• All user token balances (1B CEXT total) • Stored in internal _balances mapping • No external vault or custody layer

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
ChimeraExchangeToken.sol	transfer(to, amount)	CEXT → recipient	CEXT ← sender	Public (token holders)	• Standard ERC20 transfer • No slippage or fees
ChimeraExchangeToken.sol	transferFrom(from, to, amount)	CEXT → to address	CEXT ← from address	Public (approved spenders)	• Requires prior approval • Unlimited approvals enable total balance drain • No allowance decay mechanism

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
ChimeraExchangeToken.sol	approve(spender, amount)	–	– (sets allowance)	Public (token holders)	• Approving malicious contracts = full access • Race condition if changing non-zero allowance • Many users approve max uint256
ChimeraExchangeToken.sol	permit(owner, spender, value, deadline, v, r, s)	–	– (sets allowance)	Public (anyone with valid signature)	• Phishing attacks via malicious permit signatures • Front-running permit transactions • No signature revocation mechanism • Expired signatures still waste gas

Closing Summary

In this report, we have considered the security of CEXT. We performed our audit according to the procedure described above.

No critical issues in CEXT, just 2 issues of Informational severity were found, which have been noted and Resolved by the CEXT team.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Ouraudit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

February 2026

For

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com